

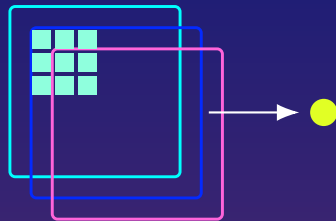
Aula 04

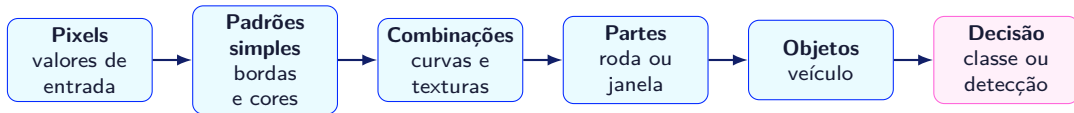
Redes Neurais Convolucionais

Como aprender características diretamente dos pixels

Dos pixels às características

Francisco Leonel Ferreira dos Santos
Treinamento de Visão Computacional com YOLO
Caxias – MA, 2026



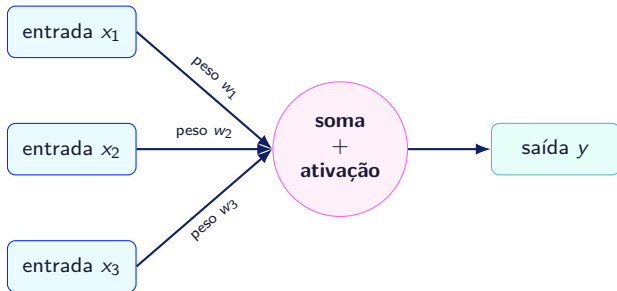


Exemplo conceitual

bordas → curvas → roda → veículo

Ideia central

A rede aprende representações úteis diretamente a partir dos exemplos.



O que ele faz?

- 1 recebe valores;
- 2 aplica pesos;
- 3 soma as informações;
- 4 aplica uma transformação;
- 5 produz uma saída.

O poder surge da combinação de muitos neurônios.

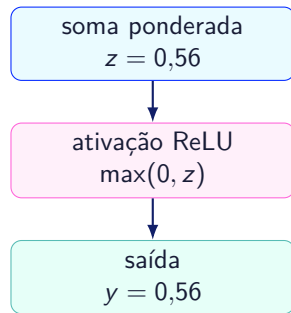
Exemplo numérico: calculando uma saída



Entradas e parâmetros

$$\begin{aligned}x_1 &= 0,7 & w_1 &= 0,8 \\x_2 &= 0,2 & w_2 &= -0,5 \\ \text{viés } b &= 0,1\end{aligned}$$

$$\begin{aligned}z &= x_1 w_1 + x_2 w_2 + b \\ &= 0,7(0,8) + 0,2(-0,5) + 0,1 \\ &= 0,56 - 0,10 + 0,10 \\ &= \boxed{0,56}\end{aligned}$$



Durante o treinamento

Os pesos e o viés são ajustados para reduzir o erro do modelo.

Pesos: quanto cada entrada influencia?

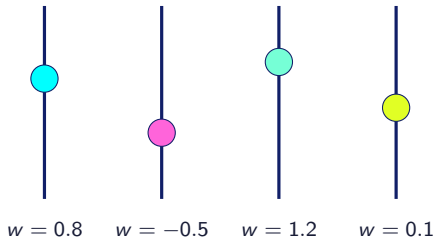


Conceito

Pesos são valores numéricos que controlam a influência das entradas e das características produzidas pela rede.

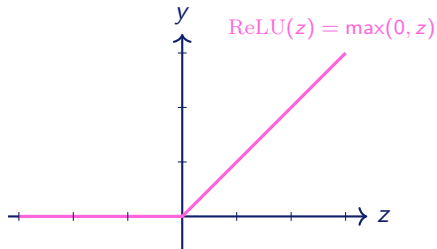
- ▶ peso positivo pode reforçar uma resposta;
- ▶ peso negativo pode reduzir uma resposta;
- ▶ magnitude maior indica influência mais intensa.

Analogia: vários controles ajustáveis



Treinar uma rede

Significa ajustar muitos parâmetros até que as previsões melhorem.

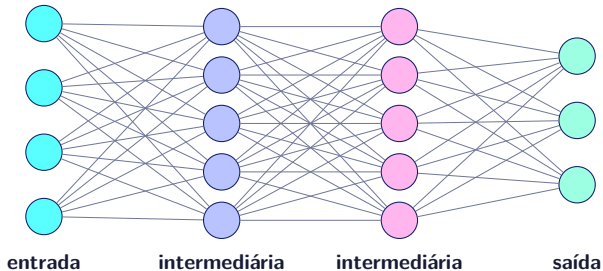


Comportamento da ReLU

- ▶ valores negativos tornam-se zero;
- ▶ valores positivos permanecem;
- ▶ a transformação permite combinar relações não lineares.

-2,0	→	0
0,0	→	0
1,7	→	1,7

A ReLU é um exemplo; redes podem utilizar outras funções de ativação.



Rede profunda

Possui diversas transformações organizadas em sequência.

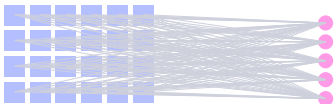
- ▶ entrada recebe dados;
- ▶ camadas internas criam representações;
- ▶ saída produz a previsão.

Por que não conectar cada pixel a tudo?



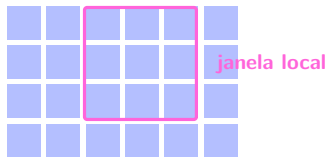
Conexão total entre pixels e neurônios

- ▶ produz um número enorme de parâmetros;
- ▶ ignora que pixels vizinhos formam padrões locais;
- ▶ dificulta reutilizar o mesmo detector em posições diferentes.



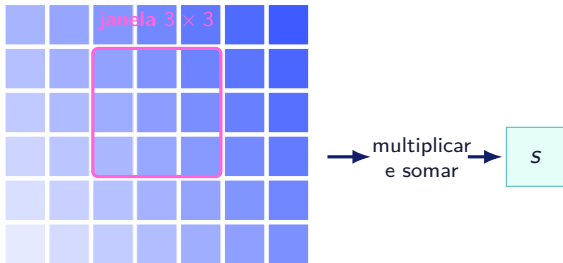
Convolução

- ▶ observa pequenas regiões locais;
- ▶ reutiliza o mesmo kernel em toda a imagem;
- ▶ preserva relações espaciais importantes.



Imagens possuem estrutura espacial; convoluções exploram essa organização.

Convolução: um kernel percorre a imagem



Em cada posição

O kernel combina os valores da região local e produz um valor no mapa de saída.

Conexão com a Aula 02

Antes escolhemos filtros manualmente. Em uma CNN, muitos filtros são ajustados durante o treinamento.

O mesmo conjunto de pesos percorre diferentes posições da imagem.

Exemplo numérico: uma janela 3×3



Região da imagem

Kernel vertical

$$\begin{bmatrix} 0 & 0 & 3 \\ 0 & 1 & 4 \\ 0 & 0 & 5 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} s &= 0(-1) + 0(0) + 3(1) \\ &\quad + 0(-1) + 1(0) + 4(1) \\ &\quad + 0(-1) + 0(0) + 5(1) \\ &= \boxed{12} \end{aligned}$$

Interpretação

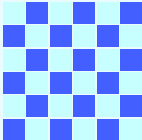
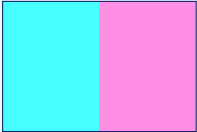
O valor alto indica uma forte mudança entre a região esquerda e a direita.

resposta
12

O símbolo $\odot$ indica multiplicação elemento a elemento.

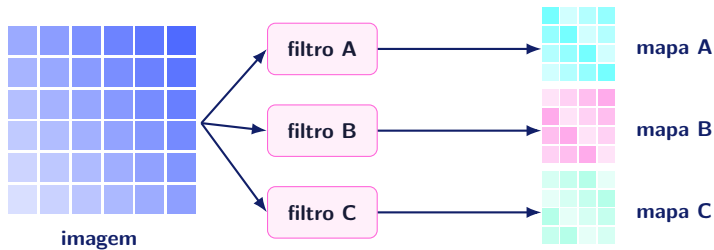
O que um filtro pode responder?



Borda vertical	Borda horizontal	Textura	Mudança de cor
$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$		
Mudança intensa entre esquerda e direita.	Mudança intensa entre parte superior e inferior.	Repetições locais e variações de superfície.	Contraste entre canais e regiões coloridas.

Na CNN, os filtros são aprendidos e podem representar padrões mais variados.

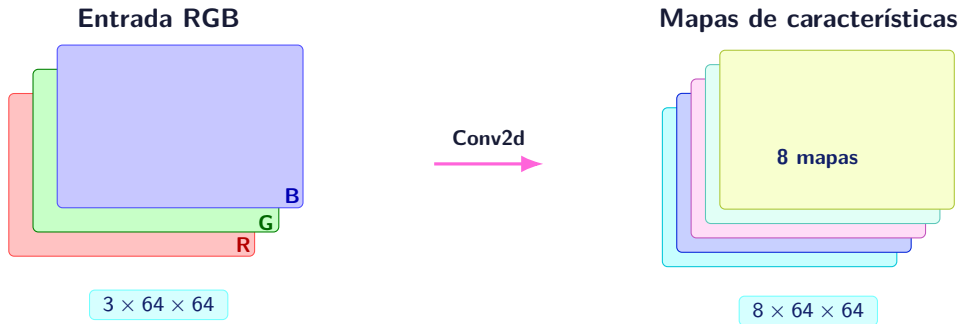
Feature map: onde um padrão produziu resposta?



Definição

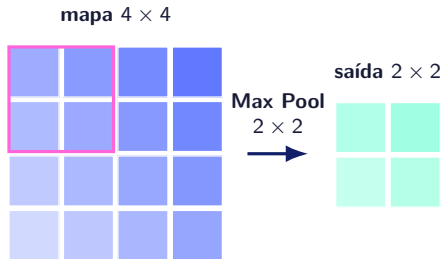
Um mapa de características registra as respostas de um filtro em diferentes posições da entrada.

De três canais de cor para muitos canais aprendidos



Mudança de significado

Canais internos passam a codificar respostas a características aprendidas, e não apenas vermelho, verde e azul.



Objetivos intuitivos

- ▶ reduzir custo computacional;
- ▶ concentrar respostas importantes;
- ▶ aumentar tolerância a pequenas mudanças locais.

Importante

Arquiteturas modernas também podem reduzir resolução usando convoluções com stride ou outras operações.

Exemplo numérico: Max Pooling 2×2



Mapa de entrada

$$\begin{bmatrix} 1 & \mathbf{5} & 2 & \mathbf{4} \\ 2 & 3 & 0 & 1 \\ 6 & \mathbf{8} & \mathbf{9} & 2 \\ 1 & 4 & 3 & 7 \end{bmatrix}$$

Os valores em negrito são os máximos das quatro janelas.

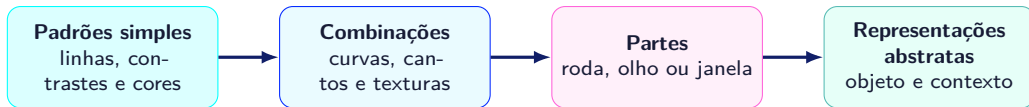
Mapa reduzido

$$\begin{bmatrix} 5 & 4 \\ 8 & 9 \end{bmatrix}$$

Primeira janela

$$\max\{1, 5, 2, 3\} = 5$$

A largura e a altura foram reduzidas pela metade.

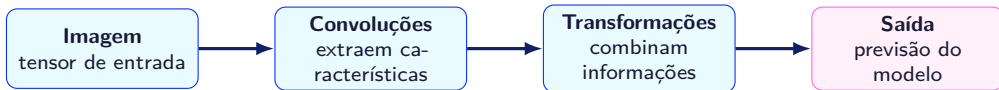


Camadas iniciais

Tendem a responder a estruturas locais mais simples e frequentes.

Camadas profundas

Combinam respostas anteriores em representações úteis para a tarefa.



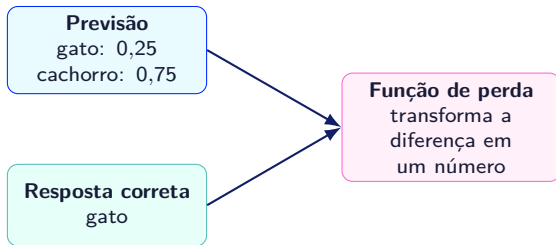
Definição

Forward pass é o percurso realizado pela entrada através das camadas para gerar uma previsão.

Durante inferência

Usamos esse caminho para prever. Durante o treinamento, a previsão também é comparada com a resposta correta.

Função de perda: quanto a previsão errou?



Papel da loss

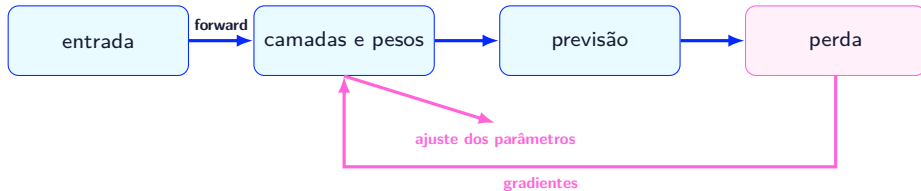
Fornecer um sinal numérico que permita avaliar e orientar o ajuste do modelo.

Analogia

É como uma nota que informa não apenas que houve erro, mas quão inadequada foi a previsão.

Treinar significa buscar parâmetros que reduzam a perda nos exemplos.

Backpropagation: usando o erro para ajustar a rede



Gradiente

Indica como a perda muda quando um parâmetro é alterado.

Backpropagation

Calcula gradientes percorrendo as operações no sentido inverso.

Resultado

Os pesos podem ser movidos em uma direção que tende a reduzir a perda.

Quem transforma gradientes em ajustes?



Gradiente

Aponta como cada parâmetro contribui para aumentar ou diminuir a perda.

Learning rate

Controla o tamanho do passo aplicado durante a atualização.

Otimizador

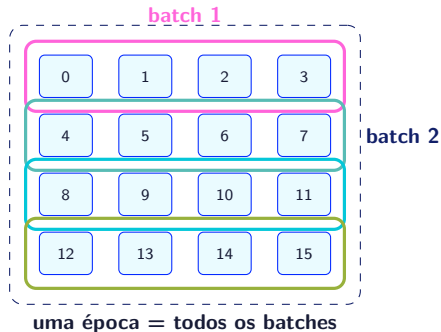
Define a estratégia usada para atualizar os parâmetros a partir dos gradientes.

$$w_{\text{novo}} = w_{\text{atual}} - \underbrace{\eta}_{\text{learning rate}} \times \underbrace{g}_{\text{gradiente}}$$

Equilíbrio

Passos muito grandes podem tornar o treinamento instável; passos muito pequenos podem deixá-lo lento.

Época e batch: como os dados percorrem o treinamento



Batch

Pequeno grupo de exemplos processado antes de uma atualização dos parâmetros.

Época

Uma passagem completa por todas as amostras do conjunto de treinamento.

Por que usar batches?

Para equilibrar memória, eficiência e frequência das atualizações.

Por que GPUs ajudam no treinamento?

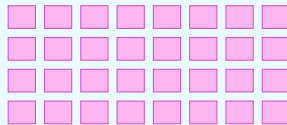


CPU: poucos núcleos mais gerais



Boa para controle, lógica e tarefas variadas.

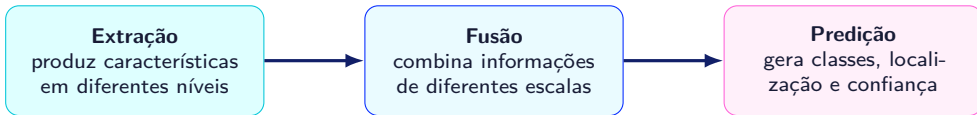
GPU: muitas unidades em paralelo



Adequada para grandes quantidades de operações numéricas semelhantes.

Na prática

É possível estudar redes pequenas usando CPU, mas imagens maiores, batches amplos e modelos profundos se beneficiam fortemente de aceleradores.



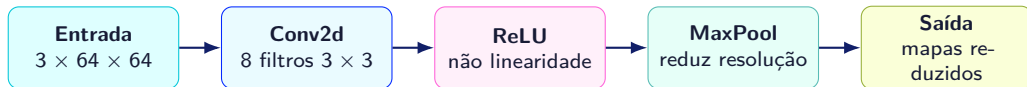
O que permanece útil?

Convolução, ativações, canais, feature maps e treinamento ajudam a interpretar o funcionamento do detector.

Arquiteturas evoluem

Diferentes versões podem combinar convoluções, atenção e outros módulos, mantendo o aprendizado profundo de representações.

Não precisamos implementar uma CNN do zero para utilizar modelos YOLO.



Antes de executar

- 1 identifique entrada e saída;
- 2 diga qual camada possui pesos;
- 3 preveja o efeito do pooling.

Pergunta

Se a convolução cria 8 filtros, quantos mapas de características devem aparecer na saída dessa camada?



```
import torch
import torch.nn as nn

conv = nn.Conv2d(
    in_channels=3,
    out_channels=8,
    kernel_size=3,
    padding=1
)

x = torch.randn(1, 3, 64, 64)
y = conv(x)

print(x.shape)
print(y.shape)
```

Parâmetros principais

- ▶ `in_channels=3`: RGB;
- ▶ `out_channels=8`: oito filtros;
- ▶ `kernel_size=3`: janela 3×3 ;
- ▶ `padding=1`: preserva altura e largura.

Observe os shapes.



Entrada

```
torch.Size([1, 3, 64, 64])
```

- 1 imagem no batch
- 3 canais RGB
- 64 altura
- 64 largura

Saída

```
torch.Size([1, 8, 64, 64])
```

- 1 exemplo processado
- 8 feature maps
- 64 altura preservada
- 64 largura preservada

A convolução mudou a quantidade de canais, mas o padding preservou a resolução.



- 1 O que são pesos?
- 2 O que é um kernel?
- 3 O que é um feature map?
- 4 O que significa treinar uma rede?
- 5 O que é uma época?
- 6 Por que CNNs são úteis para imagens?



CNNs aprendem filtros e representações que transformam pixels em características úteis para uma tarefa.

Tarefa opcional

Desenhe e explique o fluxo:

imagem → convolução → ativação → redução → características → saída

Próxima aula: Fundamentos de Detecção de Objetos.